

NFL Draft Position Prediction Based on NFL Combine Data

Michael Sloan and Alexander Shampton

GitHub Repository: <https://github.com/mpsloan/CS-7830-Final-Project>
Department of Computer Science and Engineering
Wright State University, Dayton, OH
Emails: sloan.40@wright.edu and shampton.10@wright.edu

Abstract. The NFL Draft is one of the most significant events in the professional football league year. In this paper, we investigate whether NFL Combine performance metrics can reliably predict a player's draft outcome. We separated our target as undrafted, early-round, mid-round, or late-round leveraging machine learning classification models. We evaluate two models in our experiments, a Random Forest classifier and a Multilayer Perceptron (MLP), on a dataset of 6,218 NFL Combine participants spanning 2000 to 2018. A key finding during our analysis was the discovery of target leakage through the Approximate Value (AV) feature. AV is a post-NFL performance metric, and its removal caused a 0.19 drop in macro F1, which revealed an inflation in our initial model metrics. After removing the leaking feature and optimizing both models, our best results were a macro F1 of 0.35 for Random Forest and 0.37 for MLP. Despite hyperparameter tuning, forward feature selection, and class imbalance mitigation, performance remained limited across both models. These results suggest that NFL Combine metrics alone aren't enough for draft position classification, and that meaningful improvement would require incorporating additional data sources such as collegiate performance statistics, scouting evaluations, and position-specific analysis.

Keywords: NFL Draft, NFL Combine, Random Forest classifier, Multilayer Perceptron, target leakage, class imbalance

1 Introduction

The NFL Draft garners worldwide attention with the 2025 iteration averaging 7.5 million viewers across the three days it aired, peaking at 13.6 million average viewers on Day One [1]. The main reason the NFL Draft has such great interest across the league is the effects it has on the salary cap. The NFL salary cap is the limit each team can spend on their roster for the upcoming season based on the league revenue from the year prior [2]. NFL rookies have contracts which are already accounted for based on their draft position, and due to their limited NFL experience, these contracts are generally some of the lowest on the team for their duration. This allows for teams to add high value players at extremely affordable prices for four-to-five years if they make good

selections in the draft. The NFL Draft is structured so the worst teams from the previous year, receive the highest overall pick in the draft. This is one of the pillars that keeps fan engagement in this event so high, even if your team was the worst in the league, you have the most to look forward to in the draft because your team will pick first and the player that your team chooses can turn your entire franchise around.

The NFL Combine plays a very important role in the draft process, allowing for the most prominent names to compete against each other in various on-field and off-field workouts and interviews to promote their draft position. The NFL Combine features events such as the 40-yard dash, bench press, vertical jump, broad jump, and various on field position specific drills. These results are analyzed in conjunction with film from the player’s career to slate where their draft position should be. This creates much discourse amongst the league and fans debating on player’s potential vs. their on-field production and what matters the most.

The goal of our project is to analyze a dataset containing NFL player’s combine results and using their draft position as the predicted class, classify whether they were picked early-round, mid-round, late-round, or undrafted entirely. This should provide insight on how much the NFL Combine truly matters in player evaluation to NFL scouts and general managers along with which combine drills hold the most significance. Our paper will outline our dataset and preprocessing techniques, methodologies used to interpret our data, the results of those methodologies, analysis of those results, and then a conclusion of our experiments along with the limitations we had and steps we would like to take in the future with this problem.

2 Dataset

2.1 Data Source and Statistics

We retrieved the original NFL combine dataset from a user on Kaggle [3]. The original dataset had 6,218 entries and 16 features. Amongst those entries there were 18,207 null cells which is roughly 18% of our dataset. Each entry in the dataset contained the measurables from the player’s combine workout. The specific features can be found in Table 1 below. AV is a metric developed by Pro Football Reference founder Doug Drinen to try and put a single measure on the seasonal value of any player at any position from any year [4]. This dataset ranged from players who competed in the 2000 NFL Combine through the 2018 NFL Combine.

Table 1. Original Dataset’s Features with Justification

Feature Name	Datatype	Kept or dropped	Reason
Player	string	dropped	provides no predictive insight
Position	string	kept	core player feature
Height	integer	kept	combine measurable
Weight	integer	kept	combine measurable

Forty Yard Dash	float	kept	combine measurable
Vertical Jump	float	kept	combine measurable
Bench Reps	float	kept	combine measurable
Broad Jump	float	kept	combine measurable
3-Cone Drill	float	kept	combine measurable
Shuttle	float	kept	combine measurable
Year	integer	dropped	provides no predictive insight
Pfr ID	string	dropped	provides no predictive insight
Approximate Value	float	dropped	target leaking
Team	string	dropped	provides no predictive insight
Round	integer	kept	used as target variable
Pick	integer	dropped	target leaking

2.2 Data Preprocessing

This dataset required a lot of preprocessing to take care of its null cells and remove target leakage. Our first step was dropping columns that were either insignificant or target leaking, see Table 1.

The next step in our preprocessing pipeline was encoding our one leftover categorical feature, position, using one-hot encoding. One-hot encoding was chosen because we didn't want an ordinal relationship between position groups, in the draft process positions cannot be ranked as universally more valuable than other position groups. The choice for a team to select a position group is based on their personnel on their current roster, not on the value of the actual position group. After dropping insignificant and target leaking features, our final dataset contained 33 features going into our models as listed in Table 2.

Table 2. Final set of features after preprocessing (33 total features)

Feature Name	Datatype
Height	integer
Weight	integer
Forty Yard Dash	float
Vertical Jump	float
Bench Reps	float
Broad Jump	float
3-Cone Drill	float
Shuttle	float
Pos_CB, Pos_DB, ...(24 total position columns)	boolean
Draft Position	integer (target)

Next, we preprocessed our target feature based on when a player was drafted (or not drafted). If the player was not drafted they were labeled a zero (undrafted), if they were drafted in the first or second round they are labeled a one (early-round pick), if they were drafted in the third or fourth round they were labeled a two (mid-round pick), and if they were drafted in the fifth, sixth, or seventh round they were labeled a three (late-round pick)

Subsequently, we filled in the missing values in our dataset with the medians for that specific feature. We chose the median as it is more robust to outliers than the mean, which is crucial for our dataset as at the combine there can be outliers from extremely low or high performers depending on the drill and position group.

As a final preprocessing step, we applied standard scaling to our feature matrix using sklearn's StandardScaler, which normalizes each feature to have a mean of zero and a standard deviation of one. This is particularly important for models sensitive to feature magnitude (e.g. the Multilayer Perceptron) and ensures that a feature like weight (200+ lbs) does not dominate a feature like 40-yard dash time (4-5 seconds) simply due to the difference in scale.

2.3 Feature Correlations

Figure 1 shows the Pearson correlation matrix for the eight core combine measurables and Draft Position. The most notable finding is that Draft Position shows near zero correlation with every combine feature. The strongest correlation being forty-yard dash time at only -0.06. This is a strong early indicator that combine metrics alone will struggle to predict draft position, foreshadowing the model performance we observed later. Among the combine features themselves, several strong correlations exist that are intuitive from a football perspective. Forty-yard dash time and weight are strongly positively correlated (0.84), meaning heavier players tend to run slower. Vertical jump and Broad Jump are strongly positively correlated (0.79) indicating that explosive athletes perform well across both jumping drills. Cone and Shuttle also show a strong positive correlation (0.82) which makes sense as both drills measure agility and change of direction. These inter-feature correlations suggest some redundancy among the combine drills, which may further foreshadow the poor prediction performance from our two models.

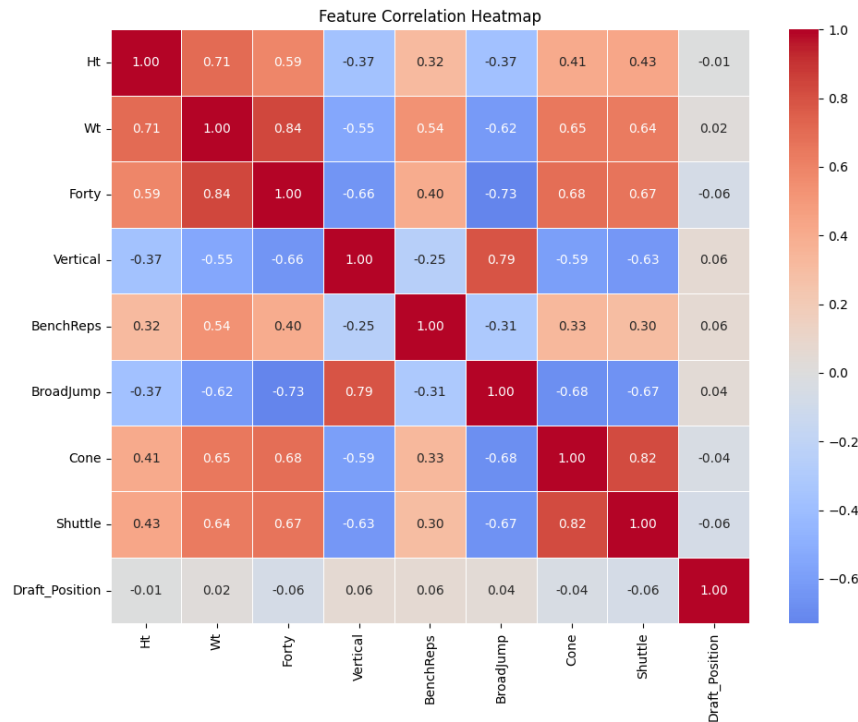


Figure 1. Feature Correlation Heatmap for the various combine measurables

2.4 Class Imbalance

As shown below in Figure 2, the dataset exhibits significant class imbalance. Undrafted players represent the largest class with nearly 2500 entries, while early round picks represent the smallest with a little over 1000 entries. This imbalance poses a challenge for classification models as they tend to bias predictions toward the majority class. To address this, we applied `class_weight='balanced'` to our Random Forest classifier and used weighted cross entropy loss in our MLP, both of which penalize the model more heavily for misclassifying minority classes.

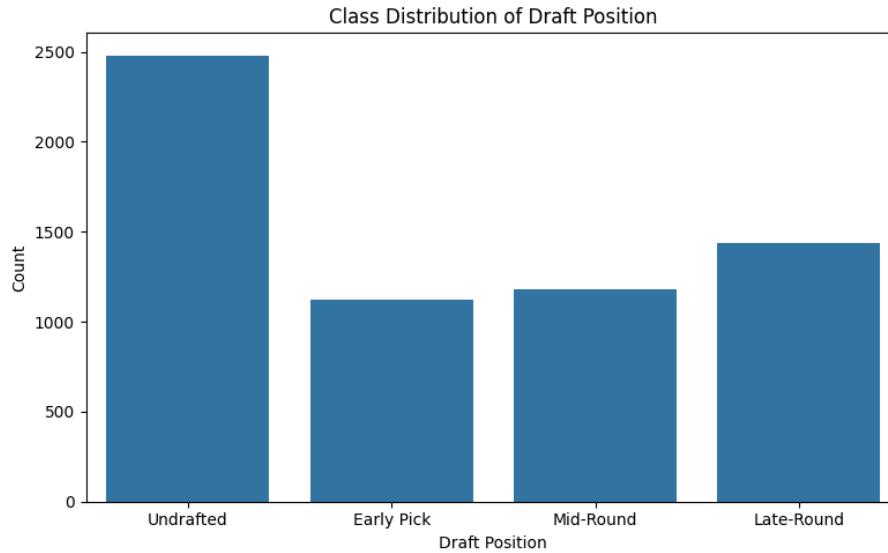


Figure 2. Bar graph showing the numbers of each class in our dataset

2.5 Forward Selection

In a separate run of the Multilayer Perceptron (MLP), we implemented forward selection. Forward selection is a greedy search algorithm that will find the most useful features for your model. Forward selection adds features one by one based on criteria like p-value or AIC. We decided to use forward selection to reduce noise and complexity of the model, which could improve our model results. In our code we utilized sklearn's SequentialFeatureSelector and chose the 12 best features from our preprocessed dataset. See Table 3 for the 12 best performing features for our model.

Table 3. 12 Best Features After Forward Selection

Feature Name	Datatype
Height	integer
Weight	integer
Forty Yard Dash	float
Vertical Jump	float
Bench Reps	float
Broad Jump	float
3-Cone Drill	float
Shuttle	float
Edge Position	boolean
Full Back Position	boolean
Offensive Tackle Position	boolean

Running Back Position	boolean
-----------------------	---------

3 Methodology

To solve the multi classification problem of predicting if or when a player will be drafted according to NFL Combine results, models need to be trained and tested with our preprocessed data. Due to the class imbalance present in our dataset, we evaluated all models primarily using macro F1-score [5], which weighs each class equally regardless of size, making it a more reliable metric than accuracy for imbalanced multiclass problems. The simple model we chose was Random Forest. We chose Random Forest because it had the best performance (specifically in macro F1) compared to the following simple models: Logistic Regression, K Nearest Neighbors, and Support Vector Classification (SVC). The complex model we chose was a Multi-Layer Perceptron (MLP)

3.1 Random Forest

The Random Forest model is similar to a decision tree, but with the addition of bagging. Bagging, or bootstrap aggregating, is an ensemble method that trains multiple decision trees on different random subsets of the training data with replacement, then aggregates their predictions through majority voting. This diversity among trees reduces variance and helps prevent overfitting compared to a single decision tree. Each tree in the forest also considers only a random subset of features at each split, controlled by the `max_features` parameter, which further decorrelates the trees and improves generalization.

To interpret which combine measurables carried the most predictive weight, we leveraged Random Forest's built-in `feature_importances_` attribute from `sklearn's RandomForestClassifier`. This attribute calculates each feature's importance based on the mean decrease in impurity (Gini impurity) across all trees in the forest. The features that consistently produce cleaner splits across more trees receive higher importance scores. As shown in Table 4, the results revealed that the 40-yard dash time was the most important feature followed closely by the athlete's weight, while all of the position indicator columns carried minimal to no predictive signal. This aligns with our central finding that combine metrics alone provide limited insight into draft position, as no single feature emerged as a dominant predictor.

Table 4. Random Forest's Selected Features and their Significance

Feature	Feature Importance
Forty Yard Dash	0.17
Weight	0.16
Bench Reps	0.10
3-Cone Drill	0.10
Vertical Jump	0.10

Broad Jump	0.10
Shuttle	0.10
Height	0.09
Positions: WR, RB, OLB, CB, DE, SS, ILB, FS, TE, QB, DT	0.01

For our Random Forest implementation, we leveraged sklearn's RandomForestClassifier with class_weight='balanced' to account for class imbalance, and performed a grid search over n_estimators, max_depth, and min_samples_split to find the optimal configuration. The grid search runs random forest by brute forcing the various hyperparameters, ensuring the most optimized set is returned.

3.2 MLP

The MLP is a supervised multi-layer neural network. MLPs contain one or more hidden layers, which can learn non-linear patterns in data. The model will learn a function given a set of features. See **Equation 1** for the function MLP will learn. The MLP needs to be designed according to our needs, where we can define the number of layers. See Figure 3 for a visualization of an MLP with one hidden layer with dimension k. We will be trying out different hidden layer architectures to develop the best model for our dataset.

$$[6] f: R^m \rightarrow R^o$$

where m is the number of dimensions for input and o is the number of dimensions for output.

Equation 1. MLP Learned Function

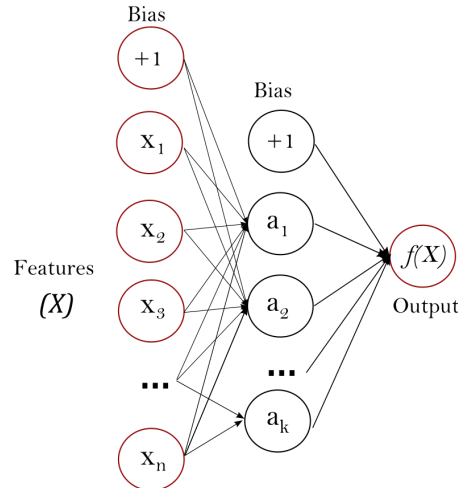


Figure 3. [6] One Hidden Layer MLP Architecture Example

To get a prediction in an MLP, a set of features need passed through the neural network. Each neuron in the hidden layer transforms the values from the previous layer with a weighted linear summation and is followed by an activation function. See **Equation 2** for a weighted linear summation for a neuron. A pass through the whole network is called a forward pass and will provide a prediction for our dataset.

$$z_j = \sum_{i=1}^m (w_{i,j} * x_i) + b_j$$

Where z_j is the weighted sum for a neuron, $w_{i,j}$ is the weight connecting input i to neuron j , x_i is value i from input features or previous activation, and b_j is the bias for neuron j .

Equation 2. Weighted Linear Summation

Our MLP was designed using torch. We designed the MLP so that each hidden layer has a linear layer and utilizing the Rectified Linear Unit (ReLU) activation function. ReLU is a mathematical function that helps neural networks learn complex patterns by using the non-negative part of its arguments. See **Equation 3** for the ReLU function. We chose ReLU because it introduces non-linearity, is computationally efficient, and has better gradients compared to other activation functions. We will be testing the following MLP architectures: 32, 64x32, 128x64x32, 100x100, 128x128x128x128.

$$f(x) = \max(0, x)$$

Equation 3. ReLU Function

4 Model Results

4.1 Random Forest Results

We will be comparing the base Random Forest model's results to the optimized hyperparameters to evaluate performance gained through tuning. As seen in Table 5, the most notable changes were constraining max_depth from unconstrained (None) to 10 and increasing min_samples_split from 2 to 10, both of which push the model toward shallower, more conservative trees. The optimized version also prefers twice the number of estimators than the base version. The max_features parameter remained unchanged at 'sqrt', suggesting this was already the optimal setting.

Table 5. Comparison of Base vs Optimized Random Forest Hyperparameters

Parameter	Base	Optimized
n_estimators	100	200
max_depth	None	10
min_samples_split	2	10
max_features	sqrt	sqrt

During our initial exploratory analysis, we leveraged Random Forest's built-in `feature_importances_` attribute to examine which features carried the most predictive weight. Approximate Value (AV) emerged as the most important feature by a significant margin. Upon further investigation we determined that AV is a post-NFL performance metric calculated after a player has already played in the league, meaning it represents information that would not be available to scouts at draft time. Including AV in our model constituted target leakage, meaning the model was effectively using the outcome of a player's NFL career to predict where they would be drafted, rendering the results misleading. As shown in Table 6 and Figure 4, the base model with AV included achieved a macro F1 of 0.52 and an overall accuracy of 0.59, with the Undrafted class achieving an F1 of 0.80 and AUC of 0.83, and Early Pick achieving an AUC of 0.75. While these numbers appear promising, they are artificially inflated by the leaking feature.

Table 6. Classification Report of Random Forest's base model with AV as a feature

	Precision	Recall	F1 Score	Support
Undrafted	0.74	0.86	0.80	496
Early Pick	0.56	0.62	0.59	224
Mid-Round	0.36	0.28	0.32	236
Late-Round	0.43	0.35	0.38	288
accuracy			0.59	1244
macro avg	0.52	0.53	0.52	1244
weighted avg	0.56	0.59	0.57	1244

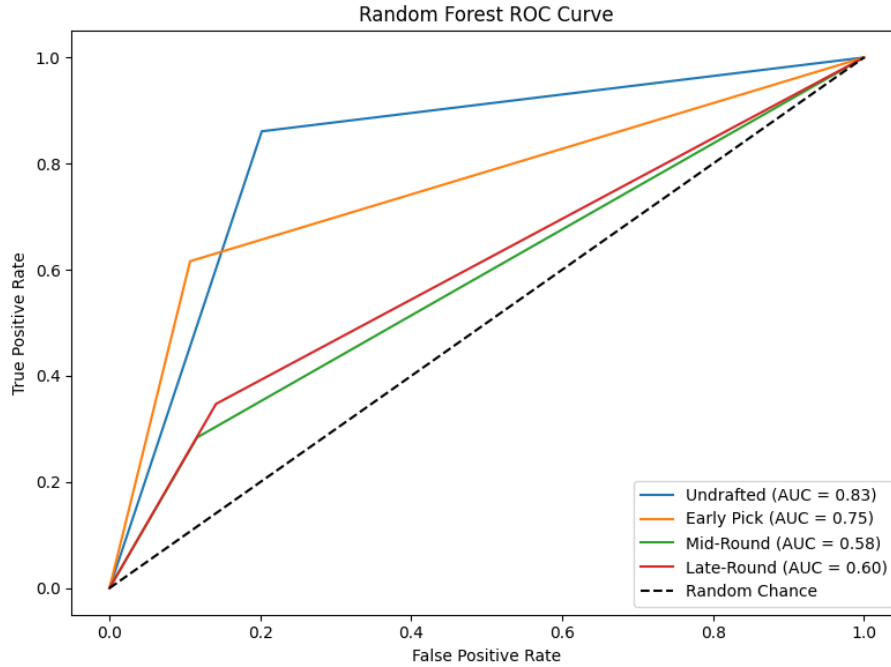


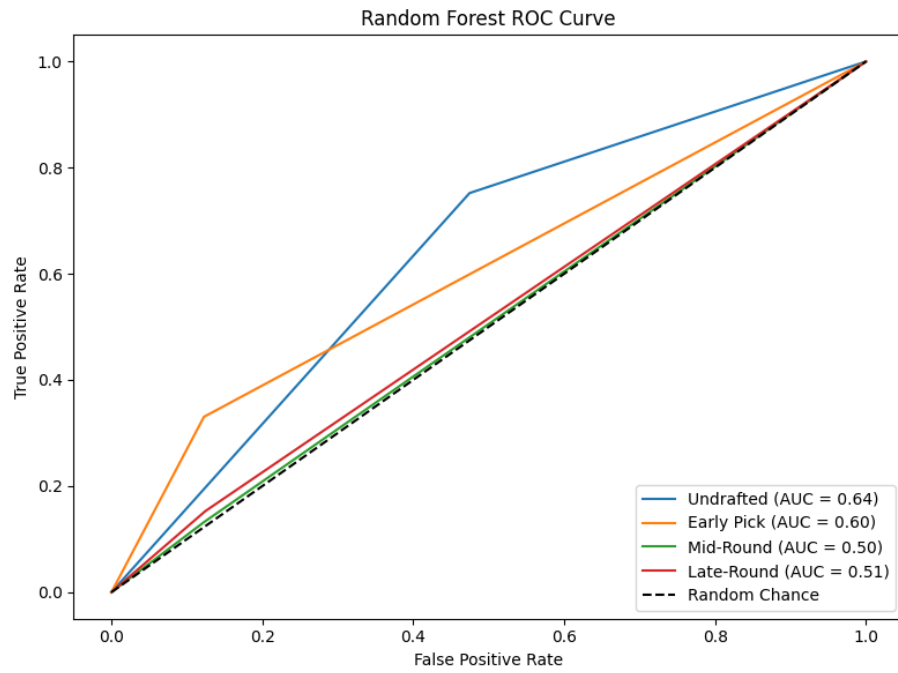
Figure 4. ROC Curve for the base Random Forest model with AV included

Upon removing AV and retraining the model exclusively on combine measurables and position indicators various metrics across all of our classes dropped drastically. This stark contrast directly quantifies how much of the model's initial performance was driven by future knowledge rather than genuine predictive signal from combine data and establishes the true performance ceiling when predicting draft position from combine metrics alone.

The results of the base Random Forest model compared to the optimized Random Forest model are shown in Tables 7 and 8 as well as Figures 5 and 6. The optimized model showed a modest improvement in macro F1 from 0.33 to 0.35, indicating that hyperparameter tuning provided marginal but measurable gains. The most notable improvement was in the Early Pick class, whose F1 score jumped from 0.35 to 0.42 and recall improved significantly from 0.33 to 0.56, suggesting the optimized model became meaningfully better at identifying early round prospects. However, this came at the cost of the Undrafted class, whose F1 dropped from 0.61 to 0.56 as the model shifted predictions away from the majority class. The ROC curves reflect a similar story with the optimized model improving Early Pick AUC from 0.60 to 0.66 and modest gains in Mid-Round and Late-Round, while the Undrafted AUC remained unchanged at 0.64. The limited overall performance gain despite tuning further supports our finding that the bottleneck is feature informativeness rather than model configuration.

Table 7. Classification Report of Random Forest's base model

	Precision	Recall	F1 Score	Support
Undrafted	0.51	0.75	0.61	496
Early Pick	0.37	0.33	0.35	224
Mid-Round	0.20	0.13	0.16	236
Late-Round	0.27	0.15	0.20	288
accuracy			0.42	1244
macro avg	0.34	0.34	0.33	1244
weighted avg	0.37	0.42	0.38	1244

**Figure 5.** ROC Curve for the base Random Forest model**Table 8.** Classification Report of Random Forest's optimized model

	Precision	Recall	F1 Score	Support
Undrafted	0.57	0.56	0.56	496
Early Pick	0.34	0.56	0.42	224

Mid-Round	0.23	0.17	0.20	236
Late-Round	0.28	0.20	0.23	288
accuracy			0.40	1244
macro avg	0.35	0.37	0.35	1244
weighted avg	0.40	0.40	0.39	1244

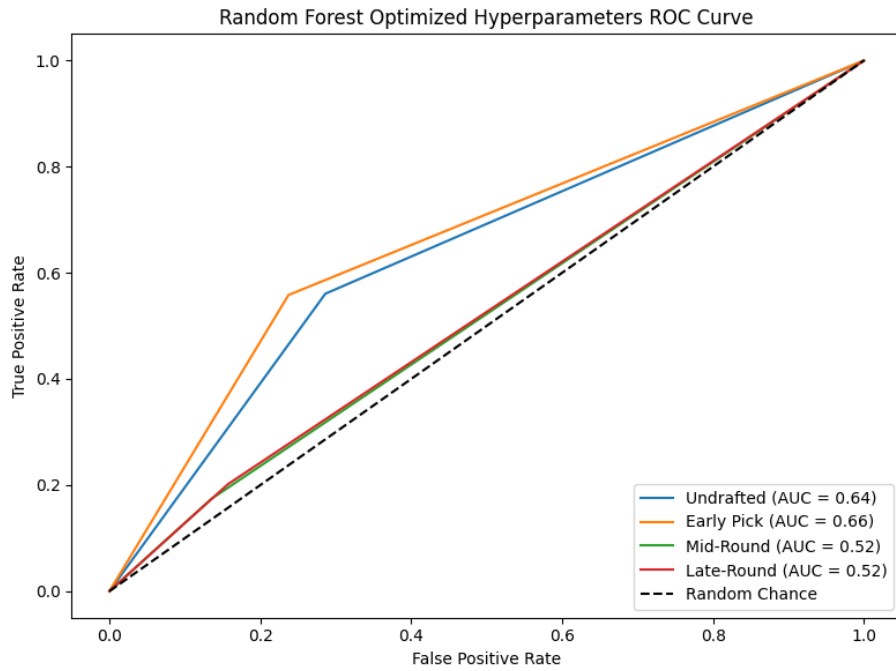


Figure 6. ROC Curve for the optimized Random Forest model

4.2 MLP Results

We ran our MLP under two conditions, one without hyper-parameter tuning and one with model tuning. Both models will be run with the following architectures: 32, 64x32, 128x64x32, 100x100, 128x128x128x128.

4.2.1 Bias MLP Results

When evaluating Random Forest and MLP model results, we discovered the results were dependent on a player's approximate value (AV). AV is a player's seasonal value from the NFL. This was a problem, since the model correlates AV with Draft pick position highly. See Table 9 for MLP results with AV included for the 32 architecture.

Table 9. 32 Architecture MLP Preliminary Results

	Precision	Recall	F1 Score	Support
Undrafted	0.70	0.74	0.72	496
Early Pick	0.58	0.67	0.62	224
Mid-Round	0.39	0.36	0.38	236
Late-Round	0.33	0.28	0.30	288
Accuracy			0.55	1244
Macro Average	0.50	0.51	0.51	1244
Weighted Average	0.53	0.55	0.54	1244

4.2.2 Base MLP Results

With our first attempt of our MLP model, we used the features in section 2.2 and utilized our base MLP model. We tested the model on each architecture and found they perform very similarly. On average accuracy was 0.43, macro-AUC was 0.57, macro average F1 was 0.33, and weighted average F1 was 0.38. We found that the 128x64x32 architecture performed slightly better with an accuracy of 0.4, 0.36 macro-average, 0.4 weighted average, and a 0.59 macro-AUC. See Figure 7 for a visual comparison of each MLP architecture. See Table 10, Figures 7, 8, and 9 for 128x64x32 architecture performance.

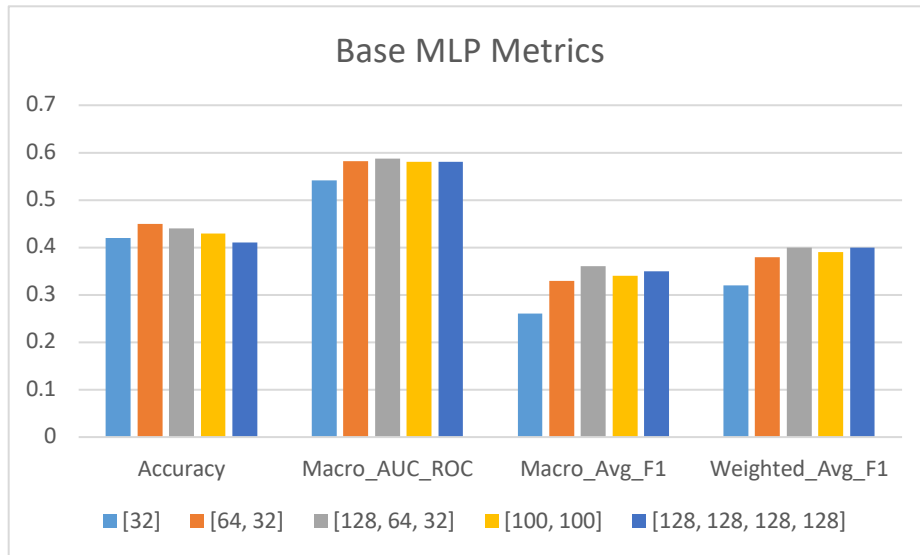
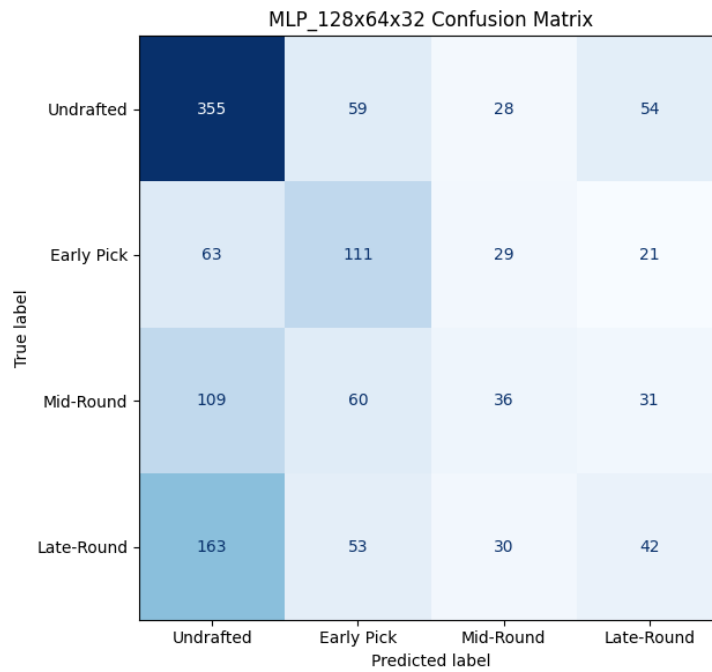
**Figure 7.** MLP Metrics

Table 10. 128x64x32 Architecture MLP Results

	Precision	Recall	F1 Score	Support
Undrafted	0.51	0.72	0.60	496
Early Pick	0.39	0.50	0.44	224
Mid-Round	0.29	0.15	0.20	236
Late-Round	0.28	0.15	0.19	288
Accuracy			0.44	1244
Macro Average	0.37	0.38	0.36	1244
Weighted Average	0.40	0.44	0.40	1244

**Figure 8.** 128x64x32 Architecture MLP Confusion Matrix

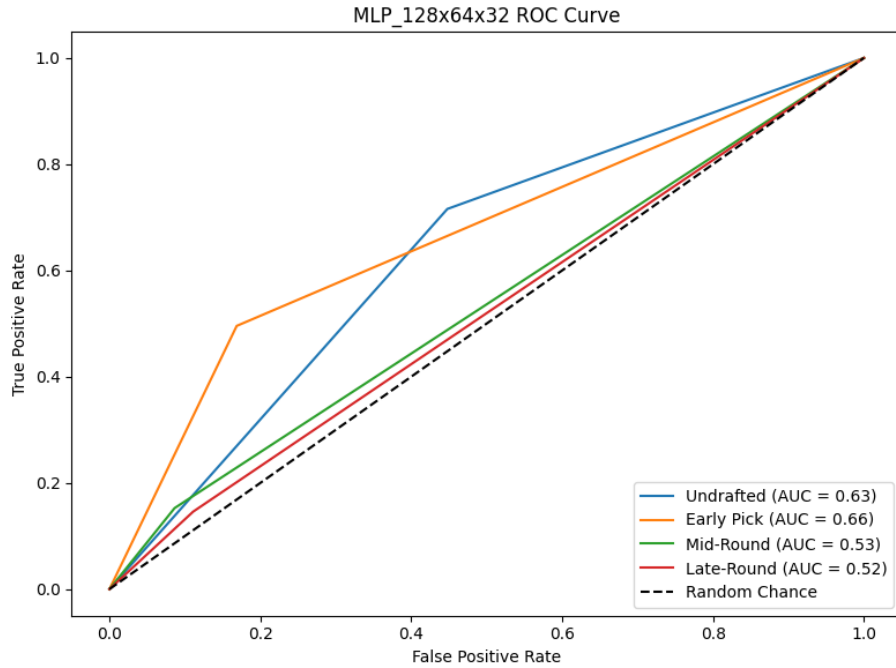


Figure 9. 128x64x32 Architecture MLP ROC Curve

4.2.3 MLP Results After Model Tuning

After analyzing the base model results, we noticed our dataset was imbalanced. We attempted to modify our model parameters and structure to help improve results. We decided to use a scheduler, which reduces the learning rate when a metric has stopped improving. Weights were added to our classes where undrafted has a 1.0 weight, early pick has a 2.2 weight, mid-round has a 2.1 weight, and late-round has a 1.7 weight. Forward selection is utilized, so we use the best features. Epochs were set to 250, drop-out was added with probability of 0.2, and our Adam optimizer has increased to a learning rate of 0.005.

Our different architectures we tested on performed very similarly. When averaging our architectures, accuracy was approximately 0.38, macro-AUC was approximately 0.55, macro average F1 was approximately 0.34, and weighted average F1 was about 0.37. See Figure 10 for a visualization of each architecture's metrics. Our best performing architecture was 32 and it had an accuracy of 0.41, a macro average f1 of 0.37, and a weighted average F1 of 0.41. See architecture 32 MLP results in Table 11, Figure 11 for the confusion matrix, and Figure 12 for the ROC curve visualization. See Figures 13 and 14 for direct comparisons of F1 scores between the base model and model with modifications.

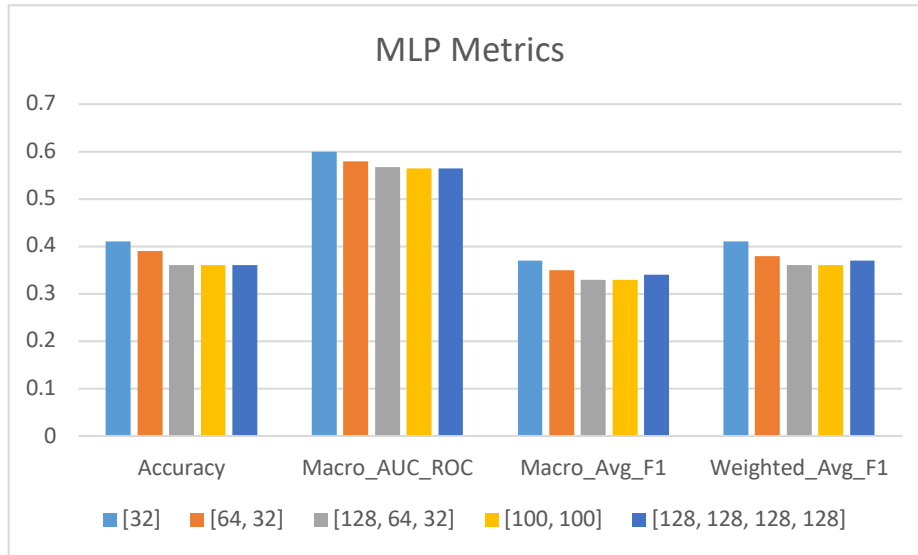


Figure 10. MLP Metrics After Model Tuning

Table 11. 32 Architecture MLP Results After Hyper-Parameter Tuning

	Precision	Recall	F1 Score	Support
Undrafted	0.61	0.53	0.56	496
Early Pick	0.34	0.61	0.44	224
Mid-Round	0.28	0.22	0.25	236
Late-Round	0.27	0.22	0.24	288
Accuracy			0.41	1244
Macro Average	0.38	0.40	0.37	1244
Weighted Average	0.42	0.41	0.41	1244

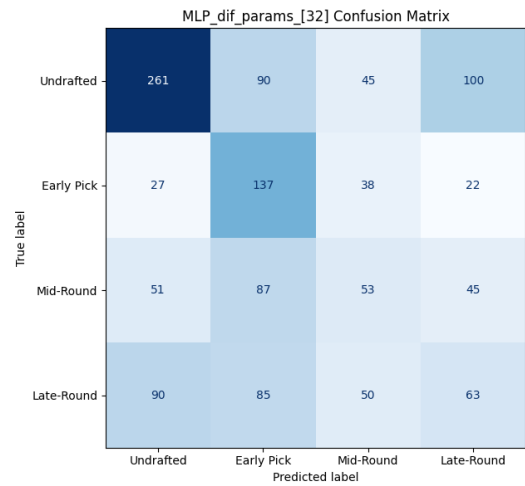


Figure 11. 32 Architecture MLP Confusion Matrix After Model Changes

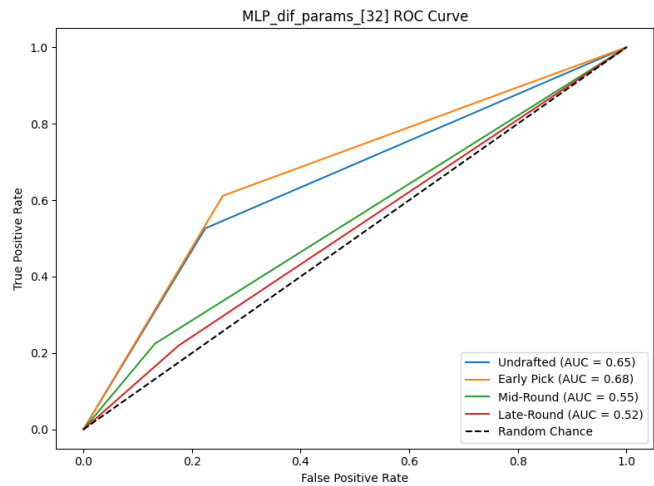


Figure 12. 32 Architecture MLP ROC Curve After Model Changes

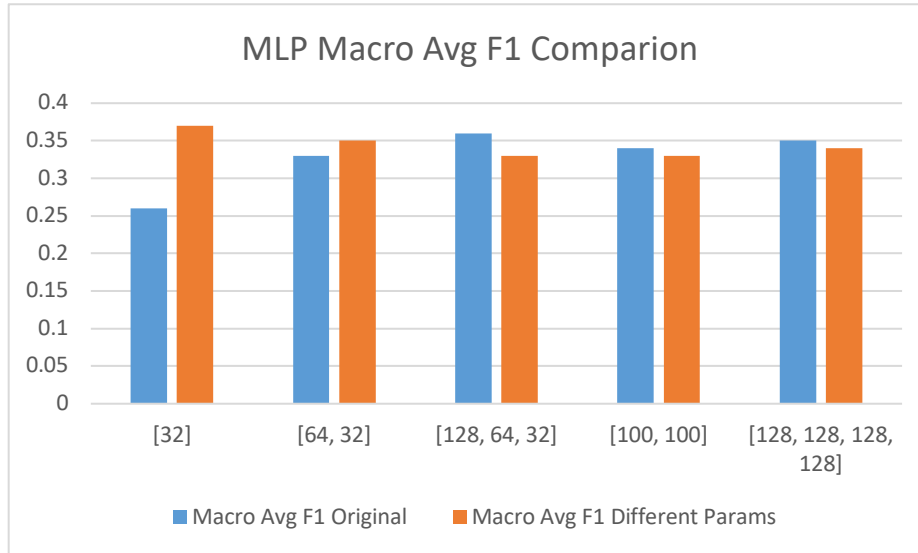


Figure 13. MLP Macro Average F1 Score Base Vs Different Parameters

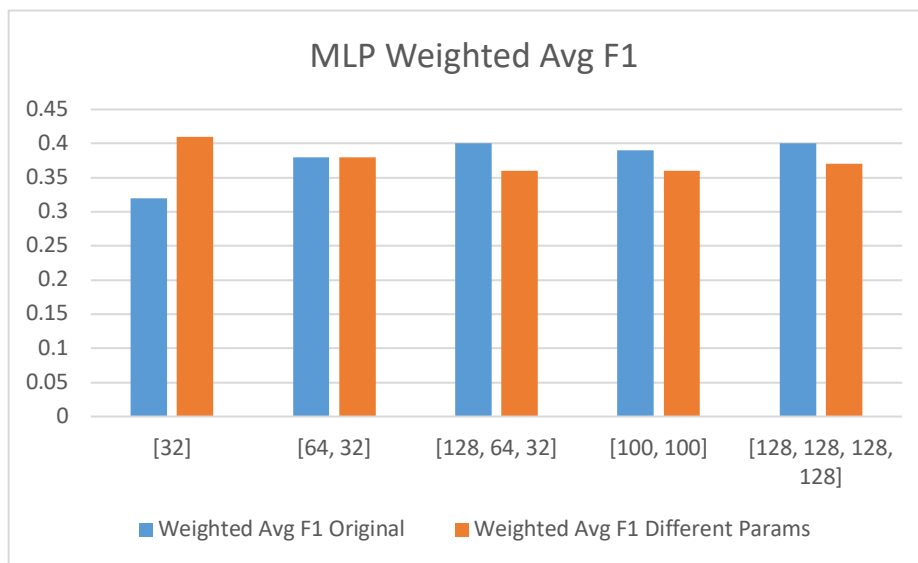


Figure 14. MLP Weight Average F1 Score Base Vs Different Parameters

5 Discussion

5.1 Deductions from Poor Model Performance

From the Exploratory Data Analysis portion of this project, we understood the problem we were undertaking would be a difficult one to solve. The near zero correlations between all combine measurables and Draft Position observed in Figure 1, foreshadowed the weak model performance that followed. No single combine feature exceeded an absolute correlation of 0.06 with Draft Position. This already suggested that the relationship between combine performance and draft outcome is far more complex than any individual combine measurable can capture. This lines up with how NFL teams evaluate prospects with combine results just being a small portion of the evaluation process. Various other factors are considered such as college film and production, medical evaluations, team needs, and interview performance. All these factors are absent in our combine dataset. Our models were expected to make a prediction on something without getting close to the full picture.

The discovery of AV as a target leaking feature was the most significant finding of our analysis. The 0.19 drop in macro F1 in the Random Forest model after removing AV from 0.52 to 0.33 directly quantifies how much of our model's initial performance was driven by future knowledge of a player's NFL career rather than genuine predictive signal from combine data.

Despite applying `class_weight='balanced'` for Random Forest and weighted cross entropy loss for our MLP, the Mid-Round class consistently proved the most difficult to predict across both models. This is a realistic finding in our experiment as mid-round picks represent the most contested territory in the draft. Mid-Round players are much more subjective as opposed to the undeniable talent within the first couple rounds of the draft. Our models struggling most with this class mirrors the uncertainty that exists in the NFL draft process itself. This is further justification that combine metrics alone are insufficient to reliably distinguish between draft tiers.

5.2 Model Comparisons

Both the Random Forest and MLP models performed similarly across all metrics, with neither model establishing a clear advantage. The optimized Random Forest achieved a macro F1 of 0.35 and weighted F1 of 0.39, while the best MLP configuration (32 architecture after model tuning) achieved a macro F1 of 0.37 and weighted F1 of 0.41, giving the MLP a marginal edge on both primary metrics.

The simplest MLP architecture of a single hidden layer with 32 neurons outperformed all deeper architectures after tuning. This is consistent with our central finding that when the underlying features carry limited predictive signal, additional model complexity does not translate to better performance. Deeper architectures with more parameters had more opportunity to overfit the noise in the combine data rather than learn meaningful patterns.

The confusion matrices further exhibit where both models struggled. In both Random Forest and MLP, the main error was misclassifying drafted players, particularly Mid-Round and Late-Round picks as Undrafted. This reflects the class imbalance in the dataset and the fact that the boundary between a player who goes undrafted and one who gets picked in a late round is difficult to differentiate from combine metrics alone. The MLP's weighted cross entropy loss did shift some predictions away from the majority Undrafted class compared to the base model, as seen in the improved Early Pick recall from 0.50 to 0.61 in the tuned 32 architecture. This, however, came at the cost of more false positives in that class.

The ROC curves tell a consistent story across both models which is that Undrafted and Early Pick consistently achieved the highest AUC values while Mid-Round and Late-Round hovered near random chance. This suggests the models learned some signal for identifying the extremes of the draft spectrum but struggled entirely with the middle classes, which are the most ambiguous and contested in real NFL draft evaluations.

Overall, we would recommend the MLP over Random Forest for this problem based on our experiments with this dataset. While performance was nearly identical on combine-only features, the MLP's ability to synthesize complex nonlinear interactions between features positions it better to take advantage of richer data sources such as collegiate statistics, where feature interactions may be more complex than Random Forest's discrete split-based approach can capture.

6 Challenges and Future Steps

Considering the poor results, specifically the accuracy and F1 scores from the various models that were evaluated. There were multiple challenges that we faced throughout this project. After reflecting on these challenges, some future steps can be arranged to improve this problem and future machine learning problems.

6.1 Creating a Balanced Dataset

A problem with this dataset is that it is highly imbalanced. In this dataset, more football players in the NFL combine did not get drafted than players who did. Experimentation with the number of players in the dataset could be attempted. It is possible if we decrease the amount of non-drafted players from our dataset, or increase the number of drafted players, our results may improve. An issue with this approach is that usually for each NFL Combine there are more players not drafted than players that are drafted. Changes like to the dataset could also be throwing away valuable information from our model. The best way forward in this is finding more data on drafted players.

6.2 Adding Additional Scouting Information

Another huge problem with this dataset, is that we are relying on combine statistics, a player's position and physical measurements. This correlation is not too strong for all positions, for example the quarter back (QB) position. A QB's combine stats are not analyzed as critically as an edge, fullback, or offensive tackle. These positions do show to be more useful features compared to the other positions according to forward selection. Scouts look at a QB's passing yards, interceptions, touchdowns, completion percentage, win loss record, and efficiency rating to determine if they want to draft them. Scouts will also look at throwing technique and decision making, but these are hard skills to quantify for a dataset. Adding a player's performance statistics may help boost our model's performance, especially for positions not reliant on the NFL Combine. Additional award metrics may be useful as well, like if a player won the Heisman. Winning an award like the Heisman is a strong indicator if a player will get drafted in the NFL.

6.3 Attempt Model for Specific Positions

Another future step we could incorporate is a model for all specific positions in the NFL Combine. Only some positions have correlation to NFL Combine statistics and NFL Draft pick. Reducing the scope of the problem to positions that correlate to these statistics could improve the model significantly. QB would be an example of what position we would potentially remove given their college statistics matter more than their combine measurables.

7 Conclusion

The results of our project were less than ideal, but through the poor performance we learned that our dataset did not provide enough valuable data to our models and our models performed similarly. Our best MLP model we found a weighted average F1 score of 0.41 and our random forest had a weighted average F1 score of 0.39. Even after our model enhancements they did not improve our base results massively. We believe if we added more valuable data to our dataset, such as college statistics, we could improve our model.

Our experimentation with different models did not yield much difference in results. The MLP did the best in model results but did not differ that much from Random Forest. We would recommend MLP over Random Forest because the MLP is designed for feature synthesis. As the data passes through hidden layers, the MLP creates new, complex combinations of our variables. Random Forest relies heavily on the features we provide it. Random Forest treats each feature as a discrete variable to split on. If we add more complex data to the dataset, like collegiate performance stats, the MLP can discover interactions that may have not been manually programmed into our feature set.

To improve future model results, research should move beyond NFL Combine metrics by integrating collegiate performance statistics and prestige awards. From our preliminary models' results with AV, we found that performance statistics can boost our model results. Additionally, addressing the class imbalance through targeted data acquisition and employing position-specific modeling will be essential for developing more robust NFL Draft forecasting tools.

Contributions

Alexander Shampton and Michael Sloan worked on this final project experiment. Alexander found the dataset, recommended what models to use, generated the MLP model, and contributed to this paper. Michael generated the Logistic Regression, K Nearest Neighbors, and Support Vector Classification (SVC) models for comparison, generated the Random Forest model, and contributed to this paper

References

- [1] NFL.com, "2025 NFL Draft is second-most watched draft ever," NFL News, Apr. 2025. [Online]. Available: <https://www.nfl.com/news/2025-nfl-draft-is-second-most-watched-draft-ever> [Accessed: Apr. 26, 2026].
- [2] D. Miller, "How does the NFL salary cap work?," Pro Football Network, Feb. 2024. [Online]. Available: <https://www.profootballnetwork.com/how-does-nfl-salary-cap-work/> [Accessed: Apr. 26, 2026].
- [3] S. Tjortjoglou, "NFL Combine Data," Kaggle Datasets, 2023. [Online]. Available: <https://www.kaggle.com/datasets/savvastj/nfl-combine-data> [Accessed: Apr. 26, 2026].
- [4] Pro-Football-Reference.com, "Approximate Value (AV) Explanation," Sports Reference. [Online]. Available: https://www.pro-football-reference.com/about/approximate_value.htm [Accessed: Apr. 26, 2026].
- [5] S. Wood, "Micro, Macro & Weighted Averages of F1 Score, Clearly Explained," Towards Data Science, Oct. 2020. [Online]. Available: <https://towardsdatascience.com/micro-macro-weighted-averages-of-f1-score-clearly-explained-b603420b292f/> [Accessed: Apr. 26, 2026].
- [6] Scikit-learn, "Neural network models (supervised)," scikit-learn.org. [Online]. Available: https://scikit-learn.org/stable/modules/neural_networks_supervised.html [Accessed: Apr. 26, 2026].